ROTEIRO DE ESTUDOS · SERVIDORES LINUX · MÓDULO 10

Virtualização (KVM) e contêineres (Podman)

KVM, QEMU e libvirt com virsh e virt-install, redes e armazenamento de VMs, cloud-init, contêineres rootless com Podman, Quadlet e systemd, LXD no Ubuntu e image mode do RHEL

Nível	Avançado
Carga sugerida	2 semanas (16–20 h)
Pré-requisitos	Módulos 04, 06 e 07; ctl01 com virtualização aninhada habilitada no hipervisor
Relação com RHCSA / LFCS	LFCS: Operations Deployment (VMs e contêineres); RHCSA: serviços iniciando no boot e gerenciamento de processos aplicados a contêineres

Objetivos do módulo

- Entender a pilha KVM + QEMU + libvirt e verificar se um host suporta virtualização.
- Criar, iniciar, parar, clonar e fazer snapshots de VMs com `virt-install` e `virsh`.
- Configurar redes e pools de armazenamento do libvirt.
- Automatizar o primeiro boot de VMs com cloud-init.
- Executar contêineres com Podman, inclusive sem root (rootless).
- Transformar contêineres em serviços com Quadlet e systemd, com atualização automática.
- Conhecer LXD (Ubuntu) e o image mode do RHEL (bootc).

1. A pilha de virtualização no Linux

Camada	Papel
KVM	Módulo do kernel que transforma o Linux num hipervisor tipo 1, usando VT-x/AMD-V
QEMU	Emula os dispositivos e executa cada VM como um processo
libvirt	API e serviço de gerenciamento (VMs, redes, armazenamento); no RHEL usa daemons modulares como `virtqemud`
Ferramentas	`virsh` (CLI), `virt-install`, `virt-manager` (gráfico), Cockpit (web, plugin `cockpit-machines`)

```
$ grep -cE 'vmx|svm' /proc/cpuinfo        # > 0: CPU com virtualização
$ lsmod | grep kvm
# dnf install qemu-kvm libvirt virt-install cockpit-machines        # [RHEL]
# apt install qemu-system-x86 libvirt-daemon-system virtinst        # [Ubuntu]
# systemctl enable --now virtqemud.socket       # [RHEL] daemon modular
# systemctl enable --now libvirtd               # [Ubuntu]
$ virt-host-validate
# usermod -aG libvirt joao                       # permite gerenciar VMs sem sudo
```

> **NOTA**
>
> O Ubuntu 26.04 oferece uma pilha de virtualização HWE opcional (`qemu-hwe`, `libvirt-hwe`, `edk2-hwe`), gerenciada pelo `ubuntu_virt_helper`, para quem precisa de versões mais novas. O RHEL 10 também tem um guia dedicado a VMs Windows.

2. Criando e gerenciando VMs

```
# virt-install --name web01 --memory 2048 --vcpus 2 \
    --disk size=20,format=qcow2 \
    --os-variant rhel10.0 \
    --cdrom /var/lib/libvirt/images/rhel-10.2-x86_64-dvd.iso \
    --network network=default --graphics none --console pty,target_type=serial \
    --boot uefi
$ osinfo-query os | grep -i rhel           # valores aceitos em --os-variant
```

Ação	virsh
Listar	`virsh list --all`
Iniciar / desligar / forçar	`virsh start web01`, `virsh shutdown web01`, `virsh destroy web01`
Iniciar com o host	`virsh autostart web01`
Console serial	`virsh console web01` (Ctrl+] sai)
Informações	`virsh dominfo web01`, `virsh domblklist web01`, `virsh domifaddr web01`
Editar o XML	`virsh edit web01`
Ajustar recursos	`virsh setvcpus`, `virsh setmem` (dentro do máximo definido)
Snapshot	`virsh snapshot-create-as web01 antes-update`, `snapshot-list`, `snapshot-revert`
Clonar	`virt-clone --original web01 --name web02 --auto-clone`
Remover	`virsh undefine web01 --remove-all-storage --nvram`

2.1 Discos

```
$ qemu-img info /var/lib/libvirt/images/web01.qcow2
# qemu-img create -f qcow2 -b base.qcow2 -F qcow2 web03.qcow2   # disco diferencial a partir de
    ↪ uma imagem-base
# qemu-img resize web01.qcow2 +10G
# virsh attach-disk web01 /var/lib/libvirt/images/dados.qcow2 vdb --persistent --subdriver qcow2
$ virsh pool-list --all; virsh vol-list default
```

qcow2 suporta provisionamento fino, snapshots e imagens-base; **raw** tem desempenho ligeiramente melhor e é mais simples.

2.2 Redes

Modo	Uso
NAT (rede `default`, 192.168.122.0/24)	VMs acessam a rede externa pelo host; ideal para laboratório
Isolada	Só VMs e host
Bridge	VM aparece na rede física como uma máquina real (bridge criada no host com nmcli ou Netplan)
macvtap	Liga a VM direto a uma placa física, sem bridge

```
$ virsh net-list --all
$ virsh net-dumpxml default
# virsh net-define lab-linux.xml && virsh net-start lab-linux && virsh net-autostart lab-linux
```

3. cloud-init

O **cloud-init** configura uma VM no primeiro boot: usuários, chaves SSH, pacotes, arquivos, comandos. É o padrão em nuvens públicas e funciona igualmente no KVM local, com imagens de nuvem (qcow2) do RHEL e do Ubuntu.

```
#cloud-config
hostname: web02
users:
  - name: joao
    groups: [wheel]
    sudo: "ALL=(ALL) NOPASSWD:ALL"
    ssh_authorized_keys:
      - ssh-ed25519 AAAA... joao@notebook
packages: [nginx, chrony]
runcmd:
  - systemctl enable --now nginx
```

```
# virt-install --name web02 --memory 2048 --vcpus 2 --import \
    --disk /var/lib/libvirt/images/web02.qcow2 --os-variant ubuntu26.04 \
    --cloud-init user-data=user-data.yaml --network network=default --graphics none
$ cloud-init status --long          # dentro da VM
# Se o osinfo instalado ainda não conhecer o ubuntu26.04, use a variante mais próxima listada por
    ↪ osinfo-query
$ cat /var/log/cloud-init-output.log
```

4. Contêineres: conceitos

Um contêiner é um **processo isolado** que compartilha o kernel do host. O isolamento vem de recursos do kernel: **namespaces** (o que o processo enxerga: PIDs, rede, montagens, usuários), **cgroups v2** (quanto ele consome) e, no RHEL, **SELinux** (o que ele pode acessar). Uma **imagem** (padrão OCI) contém o sistema de arquivos da aplicação; um **registro** distribui imagens.

	VM	Contêiner
Kernel	Próprio	Compartilhado com o host
Inicialização	Segundos a minutos	Milissegundos a segundos
Isolamento	Forte (hardware virtual)	Do kernel; depende de namespaces, cgroups e MAC
Uso típico	Sistemas completos, outros sistemas operacionais	Aplicações e microsserviços

5. Podman

O **Podman** é o motor de contêineres padrão do RHEL (e disponível no Ubuntu). Não tem daemon central, roda **sem root** com facilidade e usa a mesma sintaxe do Docker. No RHEL 10: runtime **crun** (o runc foi removido), rede **netavark**, modo **pasta** como padrão em contêineres rootless e **cgroups v2**.

```
# dnf install container-tools                  # [RHEL] podman, buildah, skopeo
# apt install podman                           # [Ubuntu]
$ podman search nginx
$ podman pull registry.access.redhat.com/ubi10/ubi
$ podman images
$ podman run -d --name web -p 8080:80 -v ~/site:/usr/share/nginx/html:Z docker.io/library/nginx
$ podman ps -a
$ podman logs -f web
$ podman exec -it web /bin/sh
$ podman inspect web | less
$ podman stop web && podman rm web
$ podman system df ; podman image prune
```

> **NOTA**
>
> A opção `:Z` no volume pede ao Podman que **rotule o diretório para o SELinux** com o contexto do contêiner. Sem ela, no RHEL, o contêiner recebe "Permission denied" ao acessar a pasta do host.

5.1 Construindo imagens

```
# Containerfile
FROM registry.access.redhat.com/ubi10/ubi-minimal
RUN microdnf install -y python3 && microdnf clean all
COPY app.py /opt/app/app.py
USER 1001
EXPOSE 8080
CMD ["python3", "/opt/app/app.py"]
```

```
$ podman build -t minha-app:1.0 .
$ podman run -d -p 8081:8080 minha-app:1.0
$ podman push minha-app:1.0 registry.linux.lab.internal/minha-app:1.0
```

5.2 Pods

Um **pod** agrupa contêineres que compartilham rede (como no Kubernetes):

```
$ podman pod create --name app -p 8080:80
$ podman run -d --pod app --name db -e POSTGRES_PASSWORD=lab docker.io/library/postgres
$ podman run -d --pod app --name web docker.io/library/nginx
$ podman pod ps
$ podman kube generate app > app.yaml        # exporta como YAML do Kubernetes
$ podman kube play app.yaml
```

6. Contêineres como serviços: Quadlet

O **Quadlet** é a forma recomendada de executar contêineres como serviços systemd: você escreve um arquivo declarativo e o systemd gera a unidade. Substitui o antigo `podman generate systemd`.

```
# ~/.config/containers/systemd/web.container      (rootless; para o sistema:
    ↪ /etc/containers/systemd/)
[Unit]
Description=Site em contêiner

[Container]
Image=docker.io/library/nginx:stable
PublishPort=8080:80
Volume=%h/site:/usr/share/nginx/html:Z
AutoUpdate=registry

[Service]
Restart=always

[Install]
WantedBy=default.target
```

```
$ systemctl --user daemon-reload
$ systemctl --user start web.service
$ systemctl --user status web
# loginctl enable-linger joao             # mantém os serviços do usuário rodando sem login aberto
$ systemctl --user enable --now podman-auto-update.timer    # atualiza imagens marcadas com
    ↪ AutoUpdate
```

7. Ubuntu: LXD e Docker

- **LXD** (instalado como snap) gerencia **contêineres de sistema** (um Ubuntu completo dentro de um contêiner) e também VMs, com a mesma interface. Ótimo para laboratórios leves.
- **Docker**: o Ubuntu fornece o pacote `docker.io` no próprio arquivo, com atualizações de segurança da distribuição. O Ubuntu 26.04 permite escolher entre trilhas de atualização regular ou mais lenta e estável para `containerd` e `runc`.

```
# snap install lxd && lxd init --auto
$ lxc launch ubuntu:26.04 teste1
$ lxc list; lxc exec teste1 -- bash
$ lxc launch ubuntu:26.04 vm1 --vm         # VM em vez de contêiner
```

8. Image mode do RHEL (bootc)

O **image mode** gerencia o próprio sistema operacional como uma imagem de contêiner: você descreve o servidor num Containerfile, constrói a imagem, e as máquinas inicializam a partir dela e se atualizam trocando de imagem de forma atômica (com retorno à versão anterior). Reduz o desvio de configuração entre servidores. É um tema avançado: conheça o conceito e explore o guia oficial depois de dominar Podman.

LABORATÓRIO

Laboratório 10.1: KVM

- No ctl01 (com virtualização aninhada), instale a pilha de virtualização e rode `virt-host-validate`.
- Crie uma VM a partir de uma imagem de nuvem do Ubuntu com cloud-init (usuário, chave e nginx).
- Faça um snapshot, quebre a VM (apague `/etc/nginx`) e reverta.
- Clone a VM e coloque o clone numa rede isolada.

LABORATÓRIO

Laboratório 10.2: Podman

- Como usuário comum, execute o nginx rootless servindo uma pasta da sua home, com `:Z`.
- Construa uma imagem própria a partir da UBI 10 para a aplicação Python do Módulo 04.
- Transforme-a em serviço com Quadlet, habilite linger e confirme que ela volta após reboot.
- Crie um pod com nginx e PostgreSQL e exporte para YAML do Kubernetes.

Checklist de revisão

- Qual o papel de KVM, QEMU e libvirt? Como verificar se o host suporta virtualização?
- Quais os comandos virsh para listar, iniciar no boot, fazer snapshot e remover uma VM com seus discos?
- Diferencie as redes NAT, isolada e bridge do libvirt.
- O que o cloud-init faz e onde ver seus logs?
- O que são namespaces e cgroups? Qual a diferença entre VM e contêiner?
- Para que serve a opção `:Z` em volumes do Podman?
- O que é o Quadlet e por que usar `loginctl enable-linger`?
- O que mudou nos contêineres do RHEL 10 (crun, netavark, pasta)?

Referências oficiais

- RHEL 10: configurar e gerenciar máquinas virtuais Linux
  https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/10/html/configuring_and_managing_linux_virtual_machines
- RHEL 10: configurar e gerenciar máquinas virtuais Windows
  https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/10/html/configuring_and_managing_windows_virtual_machines
- RHEL 10: construir, executar e gerenciar contêineres (Podman, Quadlet)
  https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/10/html/building_running_and_managing_containers
- RHEL 10: image mode
  https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/10/html/using_image_mode_for_rhel_to_build_deploy_and_manage_operating_systems
- RHEL 10: cloud-init
  https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/10/html/configuring_and_managing_cloud-init_for_rhel
- Ubuntu Server: libvirt e virsh
  https://ubuntu.com/server/docs/how-to/virtualisation/libvirt/
- Ubuntu Server: LXD
  https://ubuntu.com/server/docs/how-to/virtualisation/lxd/
- Documentação oficial do Podman
  https://docs.podman.io/

› Documentação do cloud-init
https://docs.cloud-init.io/